C4Chinese Documentation

Group Name: The BooLaGoon

Group 8 Members:
Go, Daphne Janelyn
Lim, Lanz Kendall
Tan, Tyler Justin

C4Chinese uses C and Python as its base languages, with Chinese integrated as the modified language syntax. We used C as our base language as its syntax and design rules are unambiguous. As such, C's strict enforcement of explicit delimiters, such as braces for scoping and semicolons for statement termination, ensures a clear structure that eliminates the parsing ambiguities often found in languages with implicit formatting like Python and JavaScript. However, we opt to also have in hand some of the flexibility of Python, i.e., allowing for string data type, and input & output formats. To add a little twist, we decided to incorporate Mandarin, a language we are also well familiar with, into the syntax of the tokens of our novel language.

Base Language Features

Basic Program Structure

This outlines the basic structural requirements of an executable C4 script, including the main function declaration, standard output, and standard return statements.

Code Example

```
None
// Hello World Program
hao main(){
  geini("Hello World in C4Chinese!");
  huijia 0;
}
```

1. **Comments (// or /**/)**: Comments are supported using the standard double-slash // syntax for single-line comments or /* */ syntax for multi-line comments.
2. **Entry Point (hao main())**: The execution of a C4Chinese program begins at the main() function, which is declared with the hao keyword acting as its type definition.
3. **Standard Output (geini)**: The geini() function is utilized to print string literals to the console.
4. **Return Statement (huijia)**: The huijia keyword is used to exit a function. When used inside the main() function, huijia returns an integer status code to the operating system. Conventionally, huijia 0; indicates that the program executed successfully without errors, while any non-zero value indicates an error state.

Primitive Data Types and Variables

C4Chinese supports five core primitive data types.

- **Integer (hao)**: Used to store whole numbers. Note that hao is also used to type the main() function.
- **Float (erhao)**: Used to store numbers with fractional or decimal components (e.g., 4.0).
- **Character (pinyin)**: Used to store a single character. Character values must be enclosed in single quotes (e.g., 't').
- **String (duopinyin)**: Used to store sequences of characters (text). String values must be enclosed in double quotes (e.g., "Hello").
- **Boolean (zhende)**: Used to store truth values. The accepted boolean states are zhen (representing **true**) and buzhen (representing **false**).

Variable Assignment

- **Initialization:** Variable declaration and assignment can be done in the same line (e.g., hao myInt = 4;).
- **Deferred Assignment:** Variables can be declared empty without an initial value, and then assigned a value later (e.g., hao myInt2; myInt2 = 8;).
- **Output:** All primitive variable types can be directly printed to the console using the geini() function.

```None
// Variable declaration with immediate initialization
hao myInt = 4;
erhao myFloat = 4.0;
pinyin myChar = 't';
duopinyin myStr = "4.0 pls";
zhende myBool = zhen;

// Variable declaration followed by later assignment
hao myInt2;
myInt2 = 8;
zhende myBool2;
myBool2 = buzhen;
```

Constants

Constants are declared similarly to standard variables, but they require the yongyuan keyword appended at the very beginning of the statement.

- **Initialization:** A constant must be given an initial value at the exact moment of its declaration (e.g., yongyuan hao myIntConst = 5;).
- **Strict Immutability:** Constants cannot be assigned a new value after their initial declaration. Attempting to reassign or modify a constant later in the execution flow will trigger a semantic error.
- **Standard Operations:** As long as you are only reading the data, constants can be accessed and passed to standard output functions like geini() exactly like regular variables.

```
None
// Constant declaration and initialization
yongyuan hao myIntConst = 5;
geini(myIntConst);

// Reassignment attempts will fail
// myIntConst = 10; // This line throws a semantic error
```

Note on Assignment (shi): While the = operator is often utilized for assignment, the shi and 是 alternative keywords may also be used.

Arrays

Arrays are defined by starting with the primitive type, immediately followed by the array size enclosed in square brackets, and then the identifier (e.g., hao[3] myArr1;).

- **Zero-Indexing:** C4Chinese utilizes standard zero-indexing; the first element of an array is located at index 0.
- **Access Array Elements:** Array values are retrieved using bracket notation (e.g., myArr2[4]).
- **Element-by-Element Assignment:** Variables can be declared as empty arrays first, and then populated individually by targeting specific indices (e.g., myArr1[0] = 0;).
- **List Initialization:** Arrays can be populated at the time of declaration using a comma-separated list of values enclosed within curly braces {} (e.g., hao[5] myArr2 = {1, 2, 3, 4, 5};)
- **Partial Initialization:** The language safely supports partial initialization. For example, hao[5] myArr3 = {6, 7, 8}; declares an array with a memory capacity for 5 integers, but only explicitly initializes the first 3. However, if an array was declared larger than its memory capacity, the language throws an exception.

```None
// Array declaration without immediate initialization
hao[3] myArr1;
myArr1[0] = 0;
myArr1[1] = 1;
myArr1[2] = 2;

// Array declaration with full initialization
hao[5] myArr2 = {1, 2, 3, 4, 5};

// Array declaration with partial initialization
hao[5] myArr3 = {6, 7, 8};
```

Higher-Dimensional Arrays (BONUS)

In addition to standard arrays, C4Chinese provides full support for higher-dimensional arrays.

- **Multi-Dimensional Declaration:** Higher-dimensional arrays are defined by chaining the size brackets immediately after the primitive type. For example, hao[2][2] my2DArr; declares a 2D array, while hao[2][2][2] my3DArr; declares a 3D array.
- **Nested List Initialization:** Multi-dimensional arrays can be populated at the time of declaration using nested, comma-separated lists enclosed within curly braces {} (e.g., hao[2][2] my2DArr1 = {{0, 1}, {2, 3}};).
- **Accessing Multi-Dimensional Elements:** Values are retrieved or assigned by chaining the bracket notation for each respective dimension (e.g., my3DArr[1][1][1]). C4Chinese maintains strict zero-indexing across all dimensions.
- **Element-by-Element Assignment:** Just like standard arrays, higher-dimensional arrays can be declared empty and populated individually by targeting specific coordinate indices (e.g., my2DArr2[1] = 1;).

```
None
// 2D Array declaration without immediate initialization
hao[2][2] my2DArr1;
my2DArr1[0][0] = 0;
my2DArr1[0][1] = 1;
my2DArr1[1][0] = 2;
my2DArr1[1][1] = 3;

// 2D and 3D declaration with value initialization
hao[2][2] my2DArr2 = {{0,1}, {2,3}};
hao[2][2][2] my3DArr = {{{0,1}, {2,3}}, {{4,5}, {6,7}}};
```

Pointers and Dereferencing (BONUS)

C4Chinese handles direct memory management through pointers. Unlike standard variables that store data (like a number or a string), a pointer specifically stores a **memory address**. For example, in a statement hao^ myIntPtr = &myInt; , the value of myIntPtr is the exact location in the internal memory where myInt is stored.

- **Address-Of Operator (&):** Placing an ampersand before a standard variable (e.g., &myInt) extracts and returns its internal memory address.
- **Pointer Declaration (^):** Pointer variables are declared by appending a caret (^) directly to the base data type. For example, hao^ tells the compiler, "This variable will not hold an integer; it will hold the memory address of an integer."
- **Dereference Operator (^):** Placing a caret before a pointer variable (e.g., ^myIntPtr) accesses the actual value stored at the memory address the pointer is holding. It essentially says, "Go to the address this pointer holds, and read the data there."
- **Double Pointers (^^):** C4Chinese supports variables that store the memory address of another pointer variable.
    - To declare a pointer to a pointer, stack the carets on the type declaration (e.g., hao^^).
    - To fully dereference it back to the base value, stack the carets before the variable name (e.g., ^^myIntPtrPtr2).

```
None
// Extracting an address and storing it in a pointer
hao myInt = 5;
hao^ myIntPtr = &myInt; // myIntPtr now holds a memory address, not the number
5

// Dereferencing the pointer to get the value
hao myIntDeref = ^myIntPtr;

// Double Pointers
hao myInt2 = 8;
hao^ myIntPtr2 = &myInt2;
hao^^ myIntPtrPtr2 = &myIntPtr2;

// Outputting a double-dereferenced value
geini(^^myIntPtrPtr2);
```

Records/Structs

The yinyue keyword is used to define a new struct data type. A struct acts as a blueprint that groups multiple related variables (members) of various data types under a single name. The struct definition must be enclosed in curly braces {} and terminated with a semicolon ;.

- **Instantiation:** Once defined, a struct can be instantiated by declaring the struct's name followed by a variable identifier (e.g., StudentRecord s1;).
- **Member Access (Dot Notation):** C4Chinese uses the standard dot operator (.) to access or assign values to the individual members of a struct instance. For example, s1.id = 12188888; assigns an integer to the id member of the s1 StudentRecord struct instance.
- **Nested Structs:** Structs can contain other complex data types, including arrays and even other custom-defined structs.
- **Nested Member Access:** When dealing with nested structs or arrays within structs, the dot and bracket operators can be chained together sequentially (e.g., s1.grades.grades[0] = 5; accesses the grades struct inside s1, and targets the first index of the integer array within it). Values retrieved this way can be outputted normally using geini().

```
None
// Struct Declarations
yinyue Grades {
    hao[10] grades;
};

yinyue StudentRecord {
    hao id;
    duopinyin name;
    erhao cgpa;
    Grades grades; // Nested Struct
};

// Struct Instantiation and Assignment
StudentRecord s1;
s1.id = 12188888;
s1.cgpa = 5.0;
s1.name = "Boolagoon";
s1.grades.grades[0] = 5; // Nested access
```

Typedef

The yisi keyword is utilized to define a new type alias. It maps a custom identifier to a pre-existing data type or structure (such as a primitive, a fixed-size array, or a struct). All defined typedefs act as working aliases to their actual underlying data types.

- **Usage & Instantiation:** Once declared, the alias can be used in place of the full type signature. For example, once yisi hao[3] intArr; is performed, the statement intArr myArr; is perfectly equivalent to writing hao[3] myArr;. Variables instantiated via a yisi alias function exactly like normally declared variables.

```
None
// Typedef declaration
yisi hao[3] intArr;

hao main(){
  // Using the new alias to declare an array
  intArr myArr;

  // Element assignment
  myArr[0] = 8;
  myArr[1] = 88;
  myArr[2] = 888;

  // Standard output
  geini(myArr[0]);
  geini(myArr[1]);
  geini(myArr[2]);

  huijia 0;
}
```

I/O

The geiwo() function is utilized to pause program execution and wait for the user to provide input via the console. Before geiwo() can be called, a variable must first be declared to store the incoming data, i.e., geiwo(myInt). The data provided by the user will be formatted to match the type of the destination variable.

- **Primitive Type Support:** The geiwo() function can accept and map inputs to the core primitive data types, including integers (hao), floats (erhao), strings (duopinyin), and booleans (zhende).
    - **Note on Type Resolution**: When handling complex or aliased data types (such as array elements, struct members, or custom types defined via yisi), C4Chinese will automatically resolve and map them back to their foundational primitive types (hao, erhao, pinyin, duopinyin, zhende) during processing.
- **Execution Flow:** Once the input is stored in the designated variable, the program continues its execution.

```
None
// Container variable declarations
hao inputInt;
erhao inputFloat;
duopinyin inputStr;
zhende inputBool;

// Receiving user input
geiwo(inputInt);
geiwo(inputFloat);
geiwo(inputStr);
geiwo(inputBool);

// Outputting the received values
geini(inputInt);
geini(inputFloat);
geini(inputStr);
geini(inputBool);
```

Mathematical Expressions

Arithmetic Operators: C4Chinese supports standard arithmetic operators:

- + / jia (Addition)
- - / fu (Subtraction)
- * / duojia (Multiplication)
- / / duofu (Division)
- % / xialai (Modulo/Remainder)
- ++ / jiajia (Increment)
- - - / fufu (Decrement)

Operator Precedence: The language follows standard operation precedence (PEMDAS). Parentheses () can be used to explicitly group sub-expressions and override default precedence (e.g., (1 + 1)).

Complex Operands: Mathematical expressions are fully integrated with the rest of the language's features. A single equation can seamlessly combine:

- Literal integers and standard variables (e.g., myInt * 2).
- Function calls, utilizing their returned values (e.g., add(simpleExpr, myInt2)).
- Array elements accessed via zero-indexing (e.g., myArr[9]).
- Struct members accessed via the dot operator (e.g., myStruct.id)
- Fully dereferenced pointers, even with multiple indirection (e.g., ^^myIntPtrPtr).

Type Promotion: When an arithmetic operation is performed between an integer (hao) and a float (erhao), the compiler automatically promotes the evaluated result to a float to preserve decimal precision. Similarly, characters (pinyin) involved in math operations are promoted to integers.

The Modulo Constraint: The modulo/remainder operator (%) is strictly reserved for integers (hao). Attempting to use modulo on floats or strings will trigger a semantic error.

Null Safety: Variables currently holding a null state cannot be used in any mathematical operations (addition, subtraction, multiplication, division, or modulo).

```
None
// Basic arithmetic and operator precedence
hao simpleExpr = myInt * 2 + myInt2 / (1 + 1) % 3;

// Complex expression combining functions, arrays, and pointer dereferencing
hao complexExpr = myInt + ^^myIntPtrPtr * add(simpleExpr, myInt2) -
sub(myArr[9], 3) % 4;
```

Boolean & Logical Expressions

Relational Operators: C4Chinese evaluates numerical and variable comparisons to determine boolean values. Supported operators demonstrated include:

- > / duo (Greater Than)
- < / xiao (Less Than)
- >= / duozhege (Greater Than or Equal To)
- <= / xiaozhege (Less Than or Equal To)
- == / zhege (Equal To)
- != / buzhege (Not Equal To)

Logical Operators: Multiple boolean conditions can be chained or modified together using logical operators:

- && / HE (Logical AND): Returns true only if both operands are true.
- || / HAI (Logical OR): Returns true if at least one operand is true.
- ! / LING (Logical NOT): Reverses the boolean value of the operand.

Evaluation: Regardless of the complexity or length of the expression, the final evaluated result strictly resolves to a boolean (zhende) primitive type (either zhen or buzhen).
Precedence and Grouping: Parentheses () can be heavily nested to explicitly group logical conditions, overriding default precedence and ensuring the exact intended order of evaluation.

- **Strict Relational Typing:** Some relational comparisons (<, >, <=, >=) are strictly limited to numeric types (hao, erhao) and characters (pinyin). You cannot perform these comparisons on booleans (zhende > zhende is invalid).
- **String Comparisons:** Strings (duopinyin) can be checked for equality (==) or inequality (!=) with other strings, but relational comparisons (e.g., <, >) and cross-type comparisons with numbers or null are not allowed.
- **Null Equality:** While null cannot be evaluated relationally (e.g., null > 5 throws an error), variables *can* be checked for equality (==, !=) against null to safely verify their state.

```
None
// Simple relational expression
zhende simpleBool = x + y - 8 > 6 - y + x;

// Complex expression combining logical operators with arrays, pointers, and
functions
zhende complexBool = (x + y - 8 > 6 - y + x) && (myInt + ^^myIntPtrPtr *
add(myArr[4], 4) - sub(myArr[9], 3) % 4 > 0);

// Deeply nested multiple logical expressions
zhende multipleBool = ( ((x * y + z) > (myInt / 1000)) && (z - x <= y) ) ||
!( (x + y == z + 5) && (myInt % 2 != 0) || (x / 2 > 0) );
```

String Expressions

- **String Concatenation (+):** In the context of duopinyin, strings can be concatenated using the addition operation. Instead of performing mathematical addition, the + operator joins the sequences of strings or characters end-to-end.
- **Character Concatenation:** String variables can be combined with hardcoded character literal within the same expression. For example, injecting ' ' (a space character literal) between variables is fully supported to format sentences.
- **String Coercion:** If an addition operation (+) includes at least one string (duopinyin), C4Chinese will default to concatenation. It will automatically coerce the other numerical or character operands into strings to form a single text sequence.
- **Forbidden String Math:** Strings strictly support the + operator. Attempting to use subtraction (-), multiplication (*), division (/), or modulo (%) on a string will result in an error.
- **Null Concatenation:** Concatenating a string with a null value is explicitly forbidden to prevent the accidental generation of literal "null" text in user-facing output.

```
None
// Variable declarations
duopinyin str1 = "my";
duopinyin str2 = "string";
duopinyin str3 = "is";
duopinyin str4 = "awesome";

// Direct output of concatenated variables
geini(str4 + str1);

// Concatenating variables with string literals (spaces)
geini(str4 + " " + str1);

// Assigning concatenated results to new variables
duopinyin str5 = str2 + str3 + str4;
duopinyin str6 = str2 + " " + str3 + " " + str4;

// Outputting the new variables
geini(str5);
geini(str6);
```

Conditional Statements

- **ruguo (If Statement):** The ruguo keyword initiates a conditional branch. It evaluates the boolean expression enclosed in parentheses (). If the expression resolves to true (zhen), the code block enclosed in the following curly braces {} is executed.
- **xiayigeruguo (Else-If Statement):** To test multiple, mutually exclusive conditions, the xiayigeruguo keyword can be placed after a ruguo block. It will only evaluate its condition if all preceding ruguo and xiayigeruguo conditions are evaluated to false (buzhen).
- **xiayige (Else Statement):** The xiayige keyword acts as a catch-all fallback at the end of a conditional chain. It does not take a boolean condition. Its block executes only if every preceding condition in the chain is resolved to false.
- **Block Scoping:** Just like standard C, the code to be executed for any given branch must be enclosed within curly braces {}.
- **Condition Types:** The expressions placed inside the parentheses of ruguo and xiayigeruguo must strictly evaluate to a boolean (zhende) type, utilizing the relational and logical operators.

```None
// Retrieving variable to test
hao x;
geiwo(x);

// Primary If statement
ruguo (x % 4 == 0){
    geini("Remainder of 0!");
}

// Chained Else-If statements
xiayigeruguo (x % 4 == 1) {
    geini("Remainder of 1!");
}
xiayigeruguo (x % 4 == 2) {
    geini("Remainder of 2!");
}

// Fallback Else statement
xiayige {
    geini("Remainder of 3!");
}
```

Event-Controlled Loops (While)

- **buduanruguo (While Loop):** This keyword initiates an event-controlled loop. It evaluates a boolean condition enclosed in parentheses (). If the condition resolves to true (zhen), the code block within the curly braces {} is executed. After the block finishes, the condition is evaluated again, and the loop repeats continuously until the condition becomes false (buzhen).
- **ting (Break Statement):** When called inside a loop, the ting keyword forces the program to immediately exit the loop prematurely. Any remaining code inside the loop block is ignored, and execution resumes at the first line following the loop's closing brace.
- **buting (Continue Statement):** When called inside a loop, the buting keyword immediately skips the rest of the code in the current iteration. The execution flow jumps straight back up to re-evaluate the loop's condition for the next iteration.
- **Type Safety Constraint:** Just like ruguo conditional statements, the expression inside the buduanruguo parentheses must explicitly evaluate to a boolean (zhende) type.

```
None
// Standard While Loop
hao counter = 3;
buduanruguo (counter != 0) {
    counter = counter - 1;
    geini(counter);
}

// Using a Break Statement
counter = 5;
buduanruguo (counter != 0) {
    counter = counter - 1;
    ruguo(counter == 2){
        ting; // Exits the loop entirely
    }
    geini(counter);
}

// Using a Continue Statement
counter = 5;
buduanruguo (counter != 0) {
    counter = counter - 1;
    ruguo(counter == 3){
        buting; // Skips to the next iteration
    }
    geini(counter);
}
```

Count-Controlled Loops (For)

- **buduan (For Loop):** The buduan keyword initiates a count-controlled loop. Its syntax requires exactly three statements enclosed in parentheses (), separated by semicolons ;:
    1. **Initialization:** Declares and/or assigns the starting value of the counter variable (e.g., hao ii = 0).
    2. **Condition:** A boolean expression evaluated before every iteration. If true (zhen), the loop runs. If false (buzhen), the loop terminates (e.g., ii < 5).
    3. **Update:** An expression executed at the very end of each iteration, typically used to increment or decrement the counter (e.g., ii = ii + 1).
- **ting (Break Statement):** When triggered, it forces an immediate and complete exit from the buduan loop, bypassing any remaining iterations.
- **buting (Continue Statement):** When triggered, it immediately halts the current iteration, skips any remaining code in the block, triggers the "Update" statement, and re-evaluates the condition for the next iteration.

Type Safety & Semantic Rules

- **Loop Variable Scoping:** Variables declared directly within the initialization section of the buduan signature (like hao ii = 0) are strictly scoped to that specific loop. They cannot be accessed or referenced outside of the loop's curly braces {}.
- **Condition Constraint:** The middle conditional statement must strictly evaluate to a zhende (boolean) type.
- **Numeric Updates:** The update statement typically utilizes numerical expressions. Attempting to use increment (++) or decrement (--) operators on non-numeric types (like strings) within this block will result in a semantic error.

```
None
// Standard For Loop
buduan (hao ii = 0; ii < 5; ii = ii + 1) {
    geini(ii);
}
geini("");

// Using a Break Statement
buduan (hao ii = 0; ii < 5; ii = ii + 1) {
    ruguo(ii == 2){
        ting; // Exits the loop entirely
    }
    geini(ii);
}

// Using a Continue Statement
buduan (hao ii = 0; ii < 5; ii = ii + 1) {
    ruguo(ii == 2){
        buting; // Skips the rest of the iteration
    }
    geini(ii);
}
```

Repeat-Until (Do While)

Unlike a standard while loop, a repeat-until loop guarantees that the code block will execute at least one time before the termination condition is ever evaluated.

- **laiyibian (Repeat):** After this keyword, the code inside the following curly braces {} is executed immediately, without any initial conditional checks.
- **dao (Until):** Placed immediately after the closing curly brace of the laiyibian block, this keyword defines the loop's termination condition. It must be followed by a boolean expression enclosed in parentheses and terminated with a semicolon ;.
- **Termination Logic:** The loop will continuously repeat as long as the dao condition evaluates to false (buzhen). When the condition evaluates to true (zhen), the loop terminates.
- **ting (Break Statement):** When triggered inside the block, it forces an immediate exit from the loop, skipping the dao condition check entirely.
- **buting (Continue Statement):** When triggered, it immediately halts the current iteration and skips any remaining code in the block. The execution flow jumps straight to the dao statement to evaluate if the loop should repeat.

Type Safety & Semantic Rules

- **Condition Constraint:** The expression enclosed in the dao (...) statement must explicitly evaluate to a zhende (boolean) type.
- **Scope:** Variables declared inside the laiyibian block are scoped strictly to that block and are re-initialized on every single iteration.

```
None
hao counter = 0;

// Standard Repeat-Until Loop
laiyibian {
    counter = counter + 1;
    geini(counter);
} dao (counter == 4);

// Using a Break Statement
counter = 0;
laiyibian {
    counter = counter + 1;
    ruguo(counter == 3){
        ting; // Exits the loop entirely
    }
    geini(counter);
} dao (counter == 4);

// Using a Continue Statement
counter = 0;
laiyibian {
    counter = counter + 1;
    ruguo(counter == 2){
        buting; // Skips to the 'dao' condition check
    }
    geini(counter);
} dao (counter == 4);
```

Functions, Scope, & Recursion

1. Void Functions (No Return Value)

- **Declaration (gan):** Functions that are not expected to return any data are declared with the gan keyword.
- **Restrictions:** Because they do not produce data, gan functions cannot use the return (huijia) keyword to return a value, and their execution cannot be assigned to a variable or used within an expression.

2. Non-Void Functions (Typed Return Values)

- **Declaration:** Functions expected to return data to the caller are declared using the specific primitive data type they will return (e.g., hao, erhao, zhende).
- **The Return Statement (huijia):** A typed function must explicitly use the huijia keyword to pass the evaluated data back to the caller. The returned value must strictly match the function's declared data type.
- **Evaluation & Assignment:** The result returned by a non-void function is highly versatile. It can be:
    - Stored directly into a newly declared variable (e.g., hao myInt = giveMe1();).
    - Passed instantly as an argument to another function (e.g., geini(giveMeXD());).
    - Integrated directly into larger mathematical or logical expressions (e.g., (4 < 5) || giveMeTrue()).

Scope, Parameters, & Execution Rules

- **Parameters:** Functions can accept input values by defining variables within their declaration parentheses (e.g., hao x, hao y). Arguments passed during the function call are mapped directly to these localized variables ***(Pass by Value)***.
- **Local Scoping:** Every function call generates its own independent scope. Variables declared within a function (including its parameters) will not conflict with variables of the exact same name located in the calling scope.
- **Multiple Invocations:** If a function is called multiple times, its parameters are freshly initialized and set to the newly provided argument values upon each individual execution.
- **Recursion Support:** C4Chinese fully supports recursive properties. A function can safely call itself from within its own code block to handle iterative sequences. The compiler's scope management ensures the call stack properly tracks the localized variables and return states of each recursive layer.

Type Safety & Semantic Checks

- **No Function Overloading:** C4Chinese strictly prohibits function overloading. You cannot declare multiple functions with the same identifier, even if they have completely different parameter signatures or return types. Each function name must be entirely unique within its scope.
- **Strict Argument Matching:** When calling a function, the number of arguments provided must exactly match the number of parameters defined in the function's signature. Passing too few or too many arguments will trigger a semantic error.
- **Guaranteed Returns:** If a function is declared with a return type (non-void), the compiler ensures that a `huijia` statement is guaranteed to be reached across *all* possible execution paths. If a return statement is trapped inside an `if` block without a guaranteed fallback, the compiler will throw an error.
- **Return Location:** The `huijia` (return) keyword is strictly bound to function scopes. Attempting to use a return statement floating in the global scope outside of a function body is invalid.
- **The `main()` Exception:** Unlike user-defined functions that can be typed as `gan` (void), the designated entry point `main()` function is hardcoded by the compiler to strictly require a `hao` (integer) return type.

```
None
// 1. Void Function (No Return)
gan greetMe(){
    geini("Wassup!");
}

// 2. Non-Void Function (Typed Return)
hao min(hao x, hao y){
    ruguo(x < y){
        huijia x;
    }
    xiayige {
        huijia y;
    }
}

// 3. Recursive Function
hao fib(hao x){
    ruguo (x < 1){
        huijia 0;
    }
    xiayigeruguo (x < 3) {
        huijia 1;
    }
    xiayige {
        huijia fib(x - 1) + fib(x - 2);
    }
}


hao main(){
    greetMe();              // Void Function Call
    hao z = min(3, 6);      // Non-Void Function Call
    geini(fib(7));          // Recursive Function
    huijia 0;
}
```

Chinese Keyword Aliasing (BONUS)

In line with the original idea of C4Chinese, most keywords and operators feature multiple usable alternatives. This flexibility is designed to make the language thematically unique and adaptable to different keyboard setups. For example, the assignment operator can be written using the standard symbol =, its Pinyin spelling shi, or the native character 是.

Every alternative in the language falls under one of four distinct categories:

- **Standard:** Standard C-family programming symbols (e.g., =, ++, &&, main).
- **Pinyin:** The romanized spelling of Chinese words using the standard English alphabet (e.g., shi, jia, hao).
- **Simplified:** The standard characters used in mainland China (e.g., 是, 干, 下一个).
- **Traditional:** The complex characters used in regions like Taiwan, Hong Kong, and Macau (e.g., 是, 乾, 下一個).

The following table details all multi-variant tokens within the C4Chinese lexer, categorized by their lexical type:

Token	Standard	Pinyin	Simplified	Traditional
KW_INT		**hao**	号	號
KW_FLOAT		**erhao**	二号	二號
KW_CHAR		**pinyin**	拼音	
KW_STR		**duopinyin**	多拼音	
KW_BOOL		**zhende**	真的	
KW_RECORD		**yinyue**	音乐	音樂
KW_MAIN	**main**	**zhuyao**	主要	
TYPEDEF		**yisi**	意思	
VOID		**gan**	干	乾
LT_BOOL		**zhen / buzhen**	真 / 不真	
NULL		**meiyou**	没有	沒有
CONST		**yongyuan**	永远	永遠
IF		**ruguo**	如果	
ELSE		**xiayige**	下一个	下一個

ELIF		xiayigeruguo	下一个如果	下一個如果
WHILE		buduanruguo	不断如果	不斷如果
REPEAT		laiyibian	来一遍	來一遍
UNTIL		dao	到	
FOR		buduan	不断	不斷
RET		huijia	回家	
BREAK		ting	停	
CONT		buting	不停	
INPUT		geiwo	给我	給我
PRINT		geini	给你	給你
INC	++	jiajia	加加	
DEC	--	fufu	负负	負負
GTE	>=	duozhege	多这个	多這個
LTE	<=	xiaozhege	小这个	小這個
EQ	==	zhege	这个	這個
NEQ	!=	buzhege	不这个	不這個
ADD	+	jia	加	
SUB	-	fu	负	負
MULT	*	duojia	多加	
DIV	/	duofu	多负	多負
MOD	%	xialai	下来	下來
ASSIGN	=	shi	是	
LT	<	xiao	小	
GT	>	duo	多	
AND	&&	HE	和	
OR	||	HAI	还	還
NOT	!	LING	零	

Implementation Details

Abstract Syntax Trees (ASTs)

Intermediate representation of the source code is a heavily structured Abstract Syntax Tree (AST) generated by ANTLR4. Rather than compiling down to a lower-level bytecode immediately, C4Chinese retains this tree structure for semantic analysis, optimization, and final execution.

1. AST Generation

The AST is constructed automatically during the parsing phase. The lexer tokenizes the input stream, recognizing keywords in both Pinyin and Chinese characters, and the parser organizes these tokens into a hierarchical tree based on the C4Chinese grammar rules.

2. **Core Tree Structure**

The AST represents the logical flow and nesting of the C4Chinese code. The tree is built from several key nodes:

- **The Root Node (program)**: Every valid AST begins with the program node, which has two subnodes: a list of global declarations (mainList) and the entry point (mainFunc).
- **Declarations**: The tree captures state definitions through nodes like varDecl (variables), constDecl (constants), and recordDecl (structs/records).
- **Statements (statement)**: These nodes represent actions or control flow. The AST labels specific statement types for easy visitor traversal, including:
    - Control Flow: # StatIf, # StatWhile, # StatFor, and # StatRepeat.
    - I/O: # StatPrint for outputting data and # StatInput for receiving user input.
    - Memory Operations: # StatAssign for variable mutation.
- **Expressions (expression)**: Expressions are heavily nested in the AST to enforce order of operations and strict type separation. The tree distinguishes between numeric math (numExp), string concatenation (strExp), and boolean logic (boolExp).

3. **The AST Traversal**

C4Chinese uses a multi-pass visitor pattern to interact with the AST. The tree goes through two distinct phases:

- **Phase 1: Semantic Analysis**
  The tree is traversed to build the Symbol Table and verify type safety. The visitor checks for scope violations, invalid type assignments, and ensures that functions guarantee a return statement.
- **Phase 2: Interpreter**
  Instead of compiling the AST into machine code or bytecode, the interpreter traverses the AST, dynamically allocating simulated memory, managing the call stack, and executing statement nodes. Memory management will be discussed later.

Lexical, Parser and Semantic Checks

C4Chinese ensures code correctness before it goes through code optimization and interpreter. **This is broken down into three distinct phases: Lexical, Parser, and Semantic checking**. If any errors are detected across these phases, the compiler safely halts, preventing runtime crashes and providing actionable feedback to the developer through extensive error logging.

1. **Lexical Checking (The Tokenizer)**

As the raw source code is converted into a stream of tokens (recognizing both Pinyin and Chinese characters), the lexer actively looks for unrecognized tokens or wrongly .

C4Chinese defines explicit error-catching rules directly in the grammar.

- **Custom Error Tokens:** Rules like ERROR_ID, ERROR_FLOAT, ERROR_STRING, and ERROR_CHAR capture specific formatting mistakes (e.g., an unclosed string or an invalid variable name starting with a number).
- **Actionable Feedback:** During the visitor pass, these error tokens are caught to provide the user with highly detailed warnings, often suggesting the exact character needed to fix the typo (e.g., suggesting a missing closing quote ").
- **Hidden Errors:** A final pass is performed on the token stream to catch unclosed multi-line comments or completely unrecognized characters that fell out of the standard lexer rules.

2. **Parser Checking (Syntax Validation)**

Once tokens are generated, the Parser ensures they are arranged in the correct structure (i.e., Abstract Syntax Tree) in correspondence with the C4Chinese grammar.

- **Panic Mode Recovery:** To allow for error-recovery upon encountering a single missing parenthesis or semicolon, C4Chinese uses a panicMode recovery implementation.
- **Panic Mode Mechanics:** If an invalid expression is found inside a function or the global scope, the parser consumes tokens until it finds a safe synchronization point (like a semicolon). This allows the parser to recover and continue checking the rest of the file, surfacing multiple syntax errors in a single run rather than forcing the developer to fix them one by one.

3. **Semantic Checking & Scope Management**

We traverse the tree to verify type safety, logic, and context through the **Symbol Table**.

The Symbol Table Architecture

To enforce strict lexical scoping and type rules, C4Chinese builds a hierarchical hash map (dictionary) data structure.

- **Scopes:** Environments are represented as Scope objects. Under the hood, each scope maintains a dictionary, mapping identifiers to symbols belonging to that scope. The global_scope sits at the root. Functions and Structs manage their own internal dictionary of fields and parameters.
- **Scope Chaining:** Every scope contains a pointer to its parent. If a variable isn't found in the current dictionary, the compiler traverses up the chain until it either finds the symbol or hits the global scope.
- **Symbols (The Values):** Every named entity stored is an object. VarSymbol tracks variables, constants, and how many times they are read/written. TypeSymbol handles primitives, arrays, and pointers.

Semantic Validations Performed:

With the Symbol Table tracking the environment, the checker enforces:

- **Type Matching:** Preventing invalid assignments (e.g., assigning a string to an int array) and managing implicit casts (e.g., char to int).
- **Return Guarantees:** Ensuring that functions declaring a return type actually return a value across all possible branching paths (if/else).
- **Pointer & Struct Safety:** Validating deep chains (e.g., student.grades[0]) to ensure the fields exist and the indices are valid.

Synchronizing Anonymous Blocks (Loops & Ifs)

A major challenge in semantic checking is handling unnamed scopes like the bodies of while or for loops. Because these blocks lack identifiers, C4Chinese uses a **deterministic block-indexing system**:

1. When a new block is parsed, an anonymous Scope is appended to its parent's list of blocks.
2. During traversal, the visitor calls enter_next_block(), which uses a sequential counter to fetch the correct pre-built scope for the current block.
3. This guarantees that variables declared inside a loop or if statement remain strictly local to that block and do not leak or overwrite variables outside of its scope.

Memory Management

Because the C4Chinese runtime environment is implemented in Python, the interpreter does not directly interact with raw system memory or perform byte-level allocation (like a traditional C malloc). Instead, C4Chinese acts as a **middleman**, offloading the physical memory management to Python while simulating a controlled heap using a hashmap and integer indexing.

1. **Simulated Heap RAM**

Rather than pre-allocating a massive, contiguous byte array, the runtime simulates RAM using a Python dictionary.

- **Integer Addressing:** Memory "addresses" are simply integer keys in this dictionary, starting at an arbitrary offset (e.g., 1000) to differentiate them from actual values.
- **Value Storage:** The values stored at these keys are standard Python objects (integers, strings, booleans, etc.).

This approach allows C4Chinese to enforce pointer logic without the overhead or danger of manipulating real hardware memory.

2. **Logical Sizing and Allocation**

Because C4Chinese doesn't deal in bytes, "memory size" is measured in **slots**. The TypeManager calculates exactly how many contiguous slots an entity needs:

- **Primitives & Pointers:** Require exactly 1 slot.
- **Arrays:** Require (Element Count x Element Size) slots.
- **Structs:** Require the sum total of all their internal fields' slots.

When a variable is declared, the interpreter allocates based on the “memory size”. The memory manager reserves that exact number of sequential integer keys (e.g., keys 1000, 1001, 1002 for an array of size 3) and returns the starting base address.

3. **Environments**

While the SymbolTable tracks types and scopes during compilation, the runtime **Environment** handles the actual execution state.

- Every local scope (function or block) gets its own Environment object.
- The Environment maintains a dictionary mapping the variable's string identifier to its allocated integer base address.
- When a variable is accessed, the interpreter resolves its address from the Environment, then performs a read() or write() operation on the simulated Heap using that integer key.

4. **Handling Pointers**

Because addresses are just sequential integers, C4Chinese natively supports pointers and address manipulation:

- **The Address-Of Operator (&):** Returns the integer address attributed to the variable being referenced.
- **Pointers (^):** A pointer is just a standard variable (taking up 1 memory slot) whose *value* happens to be another integer address.

5. Contiguous Access

- **Contiguous Access (Structs):** When accessing a struct field like student.gpa, the interpreter takes the struct's base address and adds the field's pre-calculated offset to find the exact integer key in the hashmap.
- **Contiguous Access (Arrays):** When accessing an array element like grades[2], the interpreter takes the array's base address and adds 2 × (Element Size) to retrieve the correct integer key in the hashmap.

Interpreter

The interpreter traverses the optimized AST in real-time, simulating memory, managing the call stack, and executing statements node by node.

1. **Program Initialization and the main Entry Point**

Execution does not start at the top of the file and run downward. C4Chinese relies on a structured entry point.

- When the interpreter starts, it allocates memory for global variables and stores all function definitions.
- **Triggering main:** Once the global state is initialized, the interpreter actively looks for the "main" function. If found, it creates the initial runtime Environment, pushes "main" onto the call stack, and triggers the execution of the main function's AST block.

2. **Call Stack and Function Invocation**

Function calls in C4Chinese involve careful synchronization between the Symbol Table and the Runtime Environment table. When the interpreter encounters a function call, it performs a sequence of operations:

1. **Argument Evaluation**: The arguments are evaluated in the *caller's* environment before changing the scope.
2. **Context Saving**: The interpreter saves the current runtime environment and the current Symbol Table scope to safely return to them later.
3. **Scope Swapping**: It creates a brand-new Environment (to prevent local variables from going beyond its scope).
4. **Stack Tracking**: The function call is pushed onto the call stack which tracks the depth of nested calls.
5. **Execution**: The function body executes. Whether it finishes naturally or throws a custom ReturnException, the finally block guarantees that the call stack is popped and both the Environment and Symbol Table scopes are restored to the caller.

Code Optimization (BONUS)

Before the Interpreter executes the program, the Abstract Syntax Tree (AST) passes through an optimization pipeline. Because the AST is held in memory as a mutable structure, C4Chinese can physically modify the tree, removing unnecessary nodes and pre-calculating static expressions to save valuable runtime execution cycles.

Three code optimizations have been implemented: **[1] Dead Code Elimination, [2] Constant Propagation and [3] Constant Folding.**

1. **Dead Code Elimination**

This scans the program to find and remove variables and struct assignments that are never used.

- **Counter Mechanism**: During the Semantic Checking phase, every variable has a tally of how many times it is read.
- **AST Pruning**: The optimizer iterates through the Symbol Table. If it finds a variable or struct field where counter == 0 (and it is not a function parameter), it marks it as dead variable.
- **Removal**: The optimizer locates the exact ANTLR AST nodes where that variable was declared or assigned and physically detaches them from their parent nodes in the tree. The interpreter never sees these variables. It doesn't waste time evaluating their assignments, and it doesn't waste simulated heap space allocating memory for them.
- **Safety Guarantees (Exceptions to Pruning):**, C4Chinese enforces strict safety rules to prevent aggressive pruning from breaking the program:
    - **Dynamic Array Indices:** The optimizer cannot statically predict the values of dynamic variables inside array brackets (e.g., whether arr[i] targets the same memory slot as arr[p]). Therefore, if an array assignment contains an index bracket ([), and the base array is read *anywhere* in the program, the optimizer conservatively preserves the assignment.
    - **Struct Field Bubbling:** Struct reads are tracked hierarchically. If a specific assignment like student.name is never directly read, but the broader student struct is passed to a function or read elsewhere, the optimizer preserves student.name to maintain the integrity of the whole struct.
    - **Function Parameters:** Variables flagged as parameters are strictly exempt from pruning, even if they are never read inside the function body. Removing a parameter node would alter the function's signature and corrupt the AST for the caller.

2. **Constant Propagation**

The second phase tracks variables with known, unchanging values and substitutes those values downstream to eliminate memory lookups.

- **State Tracking**: When a variable or constant is assigned a deterministic value (e.g., `x = 5`), the optimizer stores that literal value directly inside the SymbolTable.
- **Variable Substitution**: Later, when the optimizer visits a variable reference, it checks the Symbol Table. If the variable has a known constant value, the optimizer physically intercepts the AST node and replaces the variable identifier with the value.
- **Hierarchical Rewriting**: Because ANTLR requires a very strict tree structure, the optimizer performs safe, hierarchical rewiring. Rather than dropping a literal into the tree, it instead first wraps the new token in the correct sequence of contexts (e.g., BoolExp4Context -> BoolExp3Context, etc.) before swapping it into the parent node.
- **Safety Exceptions**: Constant propagation is incredibly dangerous around loops and conditional branches. If a variable `x` is `0` before a `while` loop, but increments (`x++`) inside the loop, the optimizer cannot blindly replace `x` with `0` everywhere.
    - Before entering an `If`, `While`, `For`, or `Repeat` block, the optimizer scans the internal statements for any assignments, inputs, or increments.
    - Any variable that is mutated inside the block has its constant value immediately invalidated. This ensures the optimizer safely ignores it, leaving the dynamic resolution of the value to the interpreter.

3. **Constant Folding**

Constant Folding evaluates deterministic mathematical, string, and boolean expressions at compile-time.

- **Compile-Time Math**: If the optimizer encounters an expression where all operands are known literals (e.g., `5 + 10`, or `"Hello " + "World"`), it evaluates the result immediately.
- **Branch Collapse**: It performs the operation (`result += next_val` or `result = left == right`), and returns that single computed result to the parent node.
- **Used with Propagation**: By resolving variables into known literals through Constant Propagation, this allows the compiler to collapse previously dynamic expressions (e.g., reducing `x + 10` directly to `15`), significantly reducing the size and complexity of the AST prior to execution.

Short-Circuit Evaluation (BONUS)

To optimize performance and allow for safe conditional checks, C4Chinese natively supports **Short-Circuit Evaluation** for its logical operators (AND and OR)

1. Boolean OR (|| / HAI / 还) expressions

- The visitor evaluates the first condition in the chain.
- Before moving to the next operand, it checks the current result. If the result is already True (zhen), it knows the entire OR chain will ultimately evaluate to True.
- It immediately triggers a return True, entirely skipping the execution of any remaining nodes in that expression.

2. Boolean AND (&& / HE / 和) AND expressions

- Similar to OR, the visitor evaluates the first condition.
- If the result is False (buzhen), it knows the entire AND chain is guaranteed to fail.
- It immediately triggers a return False, bypassing the rest of the checks.

Language Comparison

C4Chinese is designed as an educational and conceptual programming language. It is not intended to replace high-performance compiled languages in production. Rather, it serves as a bridge for Chinese-speaking learners and compiler enthusiasts to explore low-level systems concepts (like pointers, structs, and memory layout) within a highly controlled, safe environment.

C4Chinese vs. Python

- **Similarities:** Both C4Chinese (via its interpreter) and Python execute in safe, sandboxed environments where underlying physical byte allocation is abstracted away. Both evaluate expressions dynamically at runtime.
- **The C4Chinese Difference (Explicit vs. Implicit):** Python is dynamically typed and relies entirely on implicit object references. C4Chinese is strictly **statically typed** (requiring hao/int, erhao/float). Furthermore, C4Chinese forces the programmer to interact with memory manually. To pass a value by reference in C4Chinese, you must explicitly use the address-of operator (&) and dereference it (^).

C4Chinese vs. Other Chinese-Syntax Languages (e.g., Wenyan-lang)

- **Similarities:** Both provide an accessible way to write logic using Chinese characters, breaking the English-centric barrier of modern programming.
- **The C4Chinese Difference:** Many existing Chinese-syntax projects are structurally designed as **transpilers**. A transpiler (or source-to-source compiler) is a tool that simply translates code from one language into another—for example, reading Classical Chinese and generating a standard JavaScript or Python **source code file** for the computer to run later.

  While C4Chinese is *built* using Python, it **never generates Python code**. It is a fully standalone engine with its own custom Lexer, Parser, Optimizer, and Interpreter, rather than just being a syntactic skin over another language. C4Chinese also uses modern, conversational Chinese alongside **Pinyin support** (e.g., allowing both 如果 and ruguo), making it far more accessible to beginners.

Code Execution and IDE Setup

Getting Started

To get C4Chinese up and running, navigate to the provided project folder and execute the following commands. This will generate the necessary lexer and parser files for the language, as well as install all required Python dependencies:

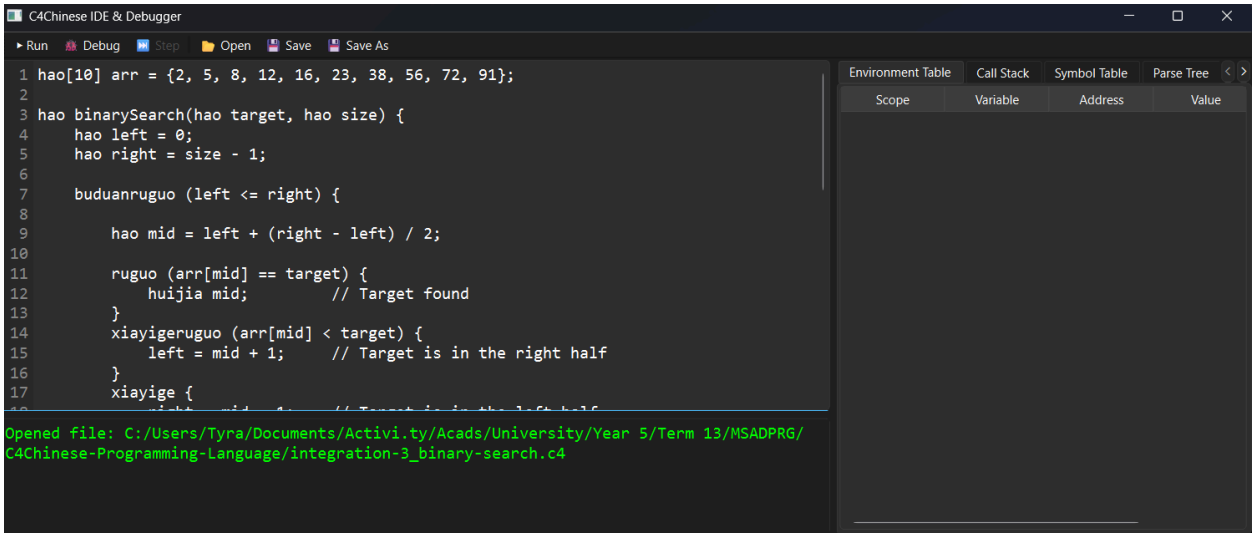
```
None
antlr4.jar -Dlanguage=Python3 C4ChineseLexer.g4
antlr4.jar -Dlanguage=Python3 -visitor -no-listener C4ChineseParser.g4
pip install -r requirements.txt
```

(Note: Depending on your system setup, you might need to run the jar files using: `java antlr4.jar ...`)

To make writing, compiling, and debugging C4Chinese a more seamless experience, we've built a custom IDE, named the ***Henhao IDE***. You can launch it by running the following command:

```
None
python IDE.py
```

Once launched, the IDE interface will open, as shown below:



IDE Layout (BONUS)

The **Henhao IDE** is split into three main resizable panels, designed to give the programmer complete visibility over the code and its underlying memory state.

- **The Code Editor:** The primary workspace on the top left. It features line numbers and an environment to write .c4 programs.

```
1 Write your C4Chinese code here...
```

- **The Interactive Console (I/O):** Located at the bottom left, this terminal acts as the main panel for I/O. It displays standard output (geini) and pauses to accept real-time user input (geiwo) directly within the text area.

```
Opened file: C:/Users/Tyra/Documents/Activi.ty/Acads/University/Year 5/Term 13/MSADPRG/
C4Chinese-Programming-Language/integration-3_binary-search.c4
```

- **The Debugger Tabs (Side Panel):** Located on the right, this panel lets the programmer inspect the underlying memory and execution context of the program. It updates in real-time during debugging and includes:
  - **Environment Table:** A live look at memory addresses, variable scopes, and nested struct/array values.
  - **Call Stack:** Tracks the current depth within recursive functions or nested calls.
  - **Symbol Table:** A static snapshot of all declared types, variables, and scopes.
  - **Parse & Optimized Trees:** Visual breakdowns of the code's Abstract Syntax Tree before and after optimization.

Debugger Panel Previews (BONUS)

Environment Table

Environment Table | Call Stack | Symbol Table | Parse Tree | Op

	Scope	Variable	Address	Value
1	Global	arr	1000	<Array[8]>
2	Global	↳ arr[0]	1000	38

Call Stack

Environment Table | Call Stack | Symbol Table | Parse Tree | Op

	Depth	Function Call
1	Level 4	mergeSort(left=0, right=1)
2	Level 3	mergeSort(left=0, right=3)
3	Level 2	mergeSort(left=0, right=7)

Symbol Table

Environment Table | Call Stack | Symbol Table | Parse Tree | Op

	Name	Type	Category	RHS Count
1	int	Unknown	PrimitiveType	Global
2	float	Unknown	PrimitiveType	Global

Parse & Optimized Trees

Table | Call Stack | Symbol Table | Parse Tree | Optimized Tree

Abstract Syntax Tree (Original)

- program
  - mainList
    - globalDecl

Base IDE Features (BONUS)

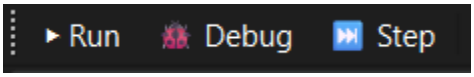
The top toolbar provides quick access to standard file operations, specifically coded to handle C4Chinese (.c4) source files:

- **Open File:** Browse the local directory to load existing .c4 scripts into the Code Editor.
- **Save:** Quickly overwrite the current file with the latest changes.
- **Save As:** Save the current editor contents as a brand new .c4 file.

Open | Save | Save As

Execution Modes: Run vs. Debug (BONUS)

The **Henhao IDE** features two modes of executing programs, allowing the programmer to execute programs continuously or analyze them line-by-line.



Run Mode (Continuous Execution)

Selecting Run executes the program from start to finish without interruption.

1. First, the IDE saves the active code to a temporary file (temp_run.c4).
2. The code passes through the Lexer, Parser, and Semantic Checkers, sequentially.
3. If an error is detected, the compilation halts and informs the programmer of the errors that must be fixed.
4. If no errors are found, the code continues through the Optimizer.
5. The runtime interpreter executes the program normally, pushing all standard output to the Interactive Console.
6. If an input (geiwo) statement is encountered, execution pauses automatically and displays a prompt in the console. Once the programmer submits the input, execution of the program simply resumes.

Debug Mode (Line-by-Line Execution)

Selecting Debug initiates an interactive debugging session. Instead of executing the program continuously, the interpreter pauses to allow for manual progression.

- **Line Highlighting:** Execution pauses at the first statement, highlighting the active line of code within the editor.
- **Live Memory Tracking:** Whenever execution pauses, the Environment Table and Call Stack dynamically refresh. This displays exactly how variables and memory addresses change during runtime.
- **Stepping Through Code:** Once Debug Mode is active, the Step button becomes available. Pressing this button advances the interpreter by exactly one statement. This enables the programmer to systematically walk through complex loops (buduan), recursive function calls, or pointer dereferences (^^) to isolate logical errors and observe data changes incrementally.